



An Internship Report
On
BACK-END AI ENGINEERING INTERNSHIP
at
FlyRank AI

Submitted
To
School of Computer Science and Engineering
IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement for the degree of
B.Tech (Computer Science and Engineering – AI/ML)

By
MAULIK PANDEY
Roll Number: 25SCS1003003495
Batch: 2025–2029

Internship Period: 1 July 2026 – 26 August 2026

1. CANDIDATE'S DECLARATION

I, MAULIK PANDEY, do hereby declare that the internship report titled “Back-End AI Engineering Internship at FlyRank AI” has been completed by me in fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (AI/ML) at IILM University.

The work presented in this report is based on my learning, assignments, development activities, and capstone project completed during my internship at FlyRank AI. Wherever information from external sources has been used, the relevant sources have been acknowledged in the references section.

I affirm that this report has not been submitted to any other institute for any other degree or diploma requirement and that I shall be responsible for any copyright or academic-integrity violation arising from material presented in this report.

Signature: _____

Student Name: MAULIK PANDEY

Roll No.: 25SCS1003003495

2. ACKNOWLEDGEMENT

I would like to express my sincere gratitude to FlyRank AI for providing me with the opportunity to undertake an eight-week internship in Back-End AI Engineering from 1 July 2026 to 26 August 2026.

I am thankful to the FlyRank team and mentors for their guidance, technical feedback, and exposure to practical software engineering workflows. The internship provided me with an opportunity to work on backend development, database systems, API design, usage metering, billing logic, testing, and integrations with external services.

I am especially grateful for the opportunity to develop and document a capstone project focused on a multi-tenant usage metering and billing engine. Working on this project helped me understand how reliability, idempotency, transactional processing, accurate monetary calculations, and webhook security are applied in backend systems.

I also express my gratitude to IILM University and the School of Computer Science and Engineering for providing the academic environment and support required to undertake this internship as part of my B.Tech programme.

Finally, I thank my family and everyone who supported and encouraged me throughout the internship.

Signature: _____

Student Name: MAULIK PANDEY

Roll No.: 25SCS1003003495

3. INTERNSHIP CONFIRMATION AND COMPLETION CERTIFICATE

This chapter contains the official internship confirmation letter and the Certificate of Completion issued by FlyRank Corp. The confirmation letter establishes the internship role and period, while the completion certificate confirms successful completion of the Backend AI Engineering Internship Program from 01.07.2026 to 26.08.2026.

Internship confirmation

Re: Internship Confirmation - Maulik Pandey, July 2026

To whom it may concern,

This letter confirms that **Maulik Pandey** (maulikpandey2007@proton.me) has been accepted into the FlyRank AI Internship program as a **Back-End AI Engineering Intern** for July 2026.

Intern name	Maulik Pandey
Role / title	Back-End AI Engineering Intern
Program	July 2026 - Backend AI Engineering
Start date	July 1, 2026
End date	August 26, 2026
Duration	8 weeks
Status	Approved

Participation confirms acceptance into the internship program and does not create employment, contractor status, wages, benefits, or a job guarantee.

Schools, universities, and employers verifying this confirmation can email verify@internship.flyrank.ai.



Alen Malkoc, Founder, CEO
FlyRank Corp.

INTERNSHIP COMPLETION CERTIFICATE

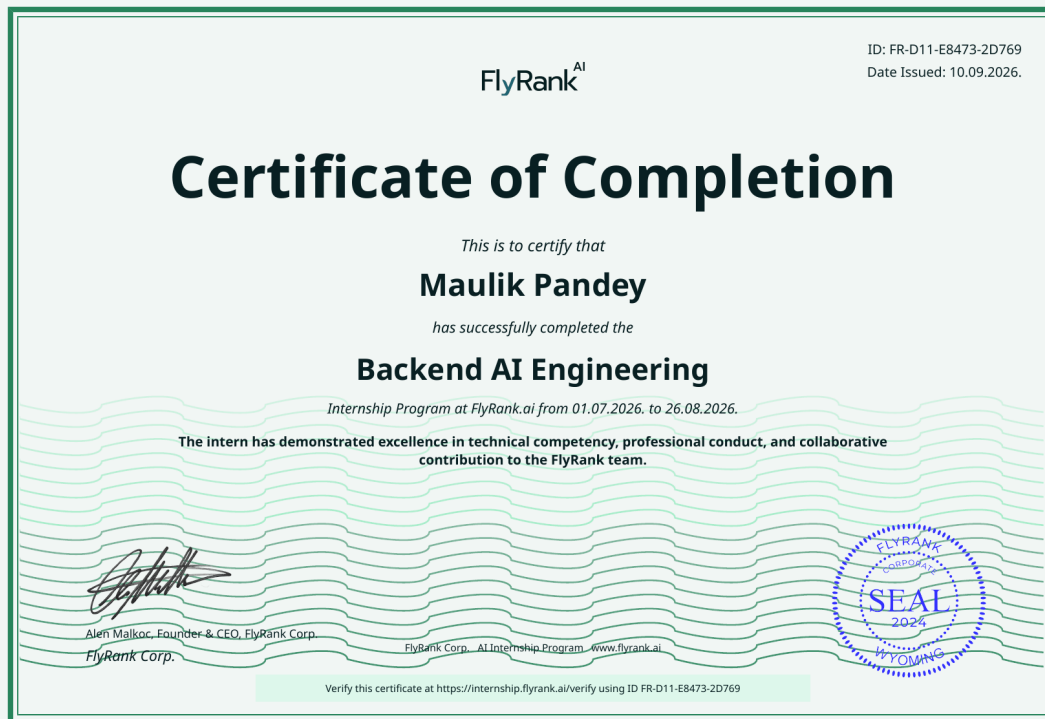


TABLE OF CONTENTS

1. Candidate's Declaration	i
2. Acknowledgement	ii
3. Internship Confirmation and Completion Certificate	iii
4. Project Description	1
4.1 Introduction	1
4.2 Organization Profile	2
4.3 Internship Role and Responsibilities	2
4.4 Problem Statement	3
4.5 Project Objectives	3
4.6 Scope of the Project	4
4.7 Technologies and Tools Used	4
4.8 System Architecture	5
4.9 Project Implementation	6
4.10 Methodology	7
4.11 Testing and Validation	8
4.12 Results and Expected Outcomes	9
4.13 Learning Outcomes	10
4.14 Project Repository and Evidence	10
5. Bibliography / References	11

4. PROJECT DESCRIPTION

4.1 Introduction

This report presents a detailed account of the eight-week Back-End AI Engineering Internship undertaken at FlyRank AI from 1 July 2026 to 26 August 2026. The internship provided practical exposure to backend engineering in an AI-oriented product environment and culminated in a capstone project focused on multi-tenant usage metering and billing.

The primary technical focus of the capstone was the design and implementation of a backend service capable of tracking API and AI-token usage, enforcing subscription quotas, calculating usage-based charges accurately, and integrating subscription state with Stripe. The project emphasized correctness and reliability through transactional processing, idempotency, webhook verification, replay protection, automated testing, and PostgreSQL persistence.

4.2 Organization Profile

FlyRank AI is an agentic organic-growth platform focused on helping brands become more visible in the modern search ecosystem. According to FlyRank's official website, its platform coordinates specialized AI agents across strategy, content production, optimization, indexation, and continuous improvement. The organization describes itself as serving D2C and SaaS brands across global markets.

FlyRank operates with a technology-oriented team that includes engineers, AI specialists, growth operators, content strategists, and client-service professionals. Its stated approach is AI-native: workflows are redesigned around what AI can enable while maintaining human oversight where it adds value.

4.3 Internship Role and Responsibilities

During the internship, I worked as a Back-End AI Engineering Intern. My responsibilities were centered on learning and applying backend engineering practices relevant to AI-enabled products. The work involved implementing APIs, organizing application logic, working with PostgreSQL, developing usage-metering and billing functionality, integrating Stripe, and validating the system through automated tests.

- Developing and structuring RESTful backend services using TypeScript and Node.js.
- Implementing request validation, service-layer business logic, repository/database access, and error handling.
- Designing usage metering for API requests and AI-token consumption.
- Implementing subscription quotas and usage-based billing calculations.
- Applying idempotency to prevent duplicate billable requests and duplicate charges.
- Working with PostgreSQL transactions, migrations, and seed data.
- Integrating Stripe Checkout and securely processing Stripe webhooks.
- Implementing webhook replay protection and subscription-state synchronization.
- Writing and executing unit and integration tests for important business rules.
- Using Git/GitHub and Docker-based workflows for development and reproducibility.

4.4 Problem Statement

AI-enabled applications often need to account for variable usage rather than a simple fixed subscription fee. A backend system therefore needs to measure consumption accurately, enforce plan limits, calculate charges deterministically, and remain correct when clients retry requests or when external payment providers resend events.

The capstone addressed this problem by building a multi-tenant metering and billing engine that records usage, enforces quotas, calculates charges using integer-cent arithmetic, handles idempotent billable requests, and synchronizes subscription information through Stripe webhooks.

4.5 Project Objectives

- Design a modular backend architecture for a multi-tenant usage metering and billing system.
- Track API-call and AI-token usage for individual tenants.
- Enforce monthly usage quotas for different subscription plans.
- Implement deterministic, exact usage-based pricing without floating-point monetary calculations.
- Prevent duplicate billing through idempotency and request fingerprinting.
- Integrate Stripe Checkout for subscription upgrades.
- Verify Stripe webhook signatures and protect against replayed events.
- Persist usage, billing, subscription, and webhook state in PostgreSQL.
- Validate critical business rules through unit and integration testing.
- Provide a reproducible development environment using Docker and documented project scripts.

4.6 Scope of the Project

The project covers the backend infrastructure required for a usage-metered SaaS billing workflow. It includes tenant-aware API processing, subscription plans, quota enforcement, usage-event recording, billable-request idempotency, token-based pricing, Stripe Checkout, webhook handling, PostgreSQL persistence, reconciliation, and automated tests.

The capstone is primarily a backend engineering project. It does not attempt to implement a complete production AI model or replace an external LLM provider. Instead, it focuses on the infrastructure needed to meter and bill AI-related usage reliably.

4.7 Technologies and Tools Used

Category	Technology	Purpose
Backend	TypeScript, Node.js, Express.js	API and server-side application development
Database	PostgreSQL, pg	Persistent relational data storage
Database tooling	Knex.js	Migrations and seed data
Payments	Stripe Checkout, Stripe Webhooks	Subscription and billing integration
Validation	Zod	Runtime request/data validation
Testing	Vitest, Supertest	Unit and API/integration testing
Development	Docker, Docker Compose	Reproducible local services and environment
Version control	Git, GitHub	Source control and project collaboration

4.8 System Architecture

The capstone follows a layered backend architecture. Incoming API requests are handled by route/controller components, which pass validated data to service-layer business logic. Repository components provide database access to PostgreSQL. Stripe is integrated as an external payment and subscription provider, while webhook events are verified and persisted before changing local subscription state.

Client / API Consumer
↓ Express Routes & Controllers Request validation and authentication context
↓ Service Layer Metering • Quota Enforcement • Billing • Idempotency
↓ Repository Layer PostgreSQL persistence • Transactions • Usage Events
↔ Stripe Integration Checkout Sessions • Signed Webhooks • Replay Protection

The architecture separates transport concerns from business rules and persistence. This makes the metering and billing logic easier to test and helps isolate external Stripe integration from the core domain logic.

4.9 Project Implementation

4.9.1 Multi-Tenant Usage Metering

The system records usage in a tenant-aware manner so that consumption is associated with the correct customer context. API requests and AI-token usage are treated as measurable billable resources.

4.9.2 Quota Enforcement

Subscription plans define monthly limits. Before accepting a billable request, the backend checks the relevant usage against the applicable quota. Requests that exceed a plan's allowance are rejected instead of silently exceeding the defined limit.

4.9.3 Idempotency and Duplicate Protection

Billing systems must tolerate retries because clients or networks may repeat requests. The project uses an idempotency key and request fingerprinting to ensure that a repeated request does not create a second billable event. Reuse of an idempotency key with a different request body is treated as a conflict.

4.9.4 Exact Billing Calculations

Monetary values are calculated using integer-based units rather than floating-point arithmetic. This avoids precision problems and makes billing deterministic. AI-token billing distinguishes the relevant token categories and applies the defined rates consistently.

4.9.5 Stripe Integration

Stripe Checkout is used for subscription upgrades. Stripe webhook requests are verified using the signed raw payload, and replay protection is applied to avoid processing the same event more than once. The local subscription state is updated only after the webhook has passed verification.

4.9.6 Database and Transactions

PostgreSQL provides durable storage for tenants, plans, subscriptions, usage events, billable requests, and webhook events. Database migrations and seed data are managed separately from runtime queries. Transactions are used where multiple state changes must succeed or fail together.

4.9.7 Testing

The project uses automated unit and integration testing to verify pricing, quota boundaries, idempotency, Stripe checkout behavior, webhook verification, replay protection, and database-backed workflows.

4.10 Methodology

The internship followed a progressive, hands-on methodology. Initial activities focused on backend fundamentals and understanding the project architecture. The later stages concentrated on implementing increasingly critical business functionality, followed by integration, testing, and final capstone validation.

Period	Focus	Activities / Outcomes
Week 1	Backend foundations	Project setup, repository structure, REST API concepts, TypeScript and layered architecture.
Week 2	Database engineering	PostgreSQL schema, migrations, seed data, repository patterns and transactional thinking.
Week 3	Usage metering	Tenant-aware usage tracking, API request metering and AI-token usage records.
Week 4	Quotas and billing	Subscription tiers, quota enforcement, deterministic usage-based pricing and integer-cent calculations.
Week 5	Reliability	Idempotency keys, request fingerprinting, duplicate prevention and failure-case handling.
Week 6	Stripe integration	Checkout sessions, subscription state, signed webhooks and replay protection.
Week 7	Testing and reconciliation	Unit/integration testing, edge cases, database-backed verification and reconciliation workflows.
Week 8	Capstone completion	End-to-end validation, documentation, cleanup, GitHub project organization and final submission.

4.11 Testing and Validation

- Verified that usage is recorded against the correct tenant and subscription context.
- Tested quota enforcement and rejection of requests exceeding allowed usage.
- Tested idempotency so that retries do not create duplicate billable requests.
- Verified conflict handling when an idempotency key is reused with a different request.
- Tested deterministic billing calculations using integer-based monetary units.
- Tested Stripe Checkout and subscription-state transitions.

- Verified webhook signature checking and rejection of forged requests.
- Tested replay protection for already-processed webhook events.
- Executed automated unit and integration tests against the backend and database.

4.12 Results and Expected Outcomes

The internship resulted in a functional backend capstone demonstrating the core requirements of a usage-metered SaaS billing system. The project combines API metering, quota enforcement, exact billing calculations, idempotency, PostgreSQL persistence, Stripe integration, webhook security, and automated testing.

- A modular and maintainable backend structure.
- Reliable measurement of API and AI-token usage.
- Subscription-aware quota enforcement.
- Deterministic billing calculations without floating-point monetary errors.
- Protection against duplicate billable requests through idempotency.
- Secure handling of Stripe webhooks with signature verification and replay protection.
- Persistent and transactionally consistent billing-related data.
- Automated verification of important business rules and failure scenarios.

4.13 Learning Outcomes

The internship significantly improved my understanding of practical backend engineering. I learned that production-oriented systems require more than implementing the happy path; correctness under retries, partial failures, duplicate events, invalid input, and external-service failures is equally important.

- Designing layered REST APIs with TypeScript and Express.js.
- Working with PostgreSQL schemas, migrations, transactions, and repository patterns.
- Understanding usage metering and subscription-based quota enforcement.
- Implementing idempotency as a reliability mechanism.
- Performing accurate monetary calculations using integer arithmetic.
- Integrating third-party payment infrastructure through Stripe.
- Securing and validating webhook-driven state changes.
- Writing automated tests for business-critical backend behavior.
- Using Docker and Git/GitHub to create reproducible and traceable development workflows.
- Approaching backend systems with an emphasis on correctness, maintainability, security, and observability.

4.14 Project Repository and Evidence

The capstone project and its implementation are maintained in a public GitHub repository:

<https://github.com/m4ulikP/flyrank-capstone-metering-billing>

The internship confirmation letter issued by FlyRank Corp. is included in Chapter 3. A separate completion certificate was not available at the time this report was prepared.

5. BIBLIOGRAPHY / REFERENCES

1. FlyRank AI, “About FlyRank,” official website, 2026.
2. FlyRank AI, “FlyRank – The Autopilot for Organic Growth,” official website, 2026.
3. FlyRank AI Internship, “Why We Are Doing This,” official internship website, 2026.
4. FlyRank AI, “The Team,” official website, 2026.
5. Maulik Pandey, “flyrank-capstone-metering-billing,” GitHub repository, 2026.
6. Node.js Documentation, <https://nodejs.org/docs/>
7. Express.js Documentation, <https://expressjs.com/>
8. PostgreSQL Documentation, <https://www.postgresql.org/docs/>
9. Stripe Documentation, <https://docs.stripe.com/>
10. TypeScript Documentation, <https://www.typescriptlang.org/docs/>
11. Docker Documentation, <https://docs.docker.com/>
12. Vitest Documentation, <https://vitest.dev/>